Reflexions- und Entwurfsdokument – IU Studien-Dashboard

Modul: Objektorientierte und funktionale Programmierung mit Python (DLBDSOOFPP01_D)

Phase: 2 – Erarbeitungs-/Reflexionsphase

Name: Sascha Schumbera

Matrikelnummer: IU14153477

Datum: April 2026

1. Umsetzung objektorientierter Konzepte in Python

Das Klassendiagramm aus Phase 1 war mein Ausgangspunkt. Für jedes OOP-Konzept habe ich ein kleines Testskript geschrieben (phase2_test_*.py), um zu prüfen, ob sich das Diagramm in Python so umsetzen lässt wie geplant – oder ob Anpassungen nötig sind. Die zentrale Frage dieser Phase war also: Was lässt sich direkt übernehmen, und wo muss ich etwas anpassen?

1.1 Enumeration – ModulStatus

Das Attribut status in Modul habe ich auf Empfehlung von Aiko Walte (Tutor) als Enumeration modelliert, damit man nicht aus Versehen falsche Begriffe oder Tippfehler beim Status eingeben kann. In Python gibt es dafür enum.Enum aus der Standardbibliothek (phase2_test_enum.py):

```
from enum import Enum

class ModulStatus(Enum):
    GEPLANT          = "Geplant"
    IN_BEARBEITUNG   = "In Bearbeitung"
    BESTANDEN        = "Bestanden"
    NICHT_BESTANDEN  = "Nicht bestanden"
```

Ergebnis: Ungültige Zuweisungen werfen automatisch einen ValueError – das muss nicht manuell abgefangen werden. Alle Zustände lassen sich über eine Schleife auflisten, was für spätere Dropdown-Menüs nützlich ist. Das Klassendiagramm konnte hier ohne Änderungen übernommen werden.

1.2 Kapselung – @property statt Getter/Setter

Im Diagramm sind alle Attribute als privat markiert. Da Python keinen echten Schreibschutz für Variablen hat, habe ich einen Unterstrich vor den Namen gesetzt (z. B. _note) – das ist eine Konvention, die signalisiert: nicht direkt von außen ändern. Für den Zugriff habe ich @property verwendet statt klassische get/set-Methoden (phase2_test_property.py):

```
class Pruefungsleistung:
    @property
    def note(self) -> float:
        return self._note

    @note.setter
    def note(self, wert: float) -> None:
        if not 1.0 <= wert <= 5.0:
            raise ValueError(f"Note {wert} ungültig (1.0-5.0)")
        self._note = wert
```

Ergebnis: @property ist ein Implementierungsdetail und erscheint nicht im UML. Die Validierung sitzt direkt im Setter – ich muss also nicht an jeder Stelle im Code extra prüfen, ob eine Note gültig ist.

Information Expert Prinzip: Aiko Walte (Tutor) hat darauf hingewiesen, dass get_note() zur Klasse Pruefungsleistung gehört, weil dort die Note gespeichert ist. Die Note im Modul abzufragen wäre falsch – das Modul hat sie gar nicht. In der Umsetzung ist note deshalb als @property in Pruefungsleistung untergebracht.

1.3 Vererbung – Abschlussarbeit erbt von Modul

Vererbung wird in Python durch Angabe der Elternklasse in Klammern umgesetzt. Mit super().__init__() werden die gemeinsamen Attribute an Modul weitergegeben – so muss der Konstruktor von Modul nicht doppelt geschrieben werden (phase2_test_vererbung.py):

```
class Abschlussarbeit(Modul):
    def __init__(self, thema: str, betreuer: str) -> None:
        super().__init__("Bachelorarbeit", "DLBABSCHLUSSB01", ects=12)
        self._thema = thema
        self._betreuer = betreuer
```

Ergebnis: Im Test erzeuge ich ein Abschlussarbeit-Objekt (ba) und prüfe mit isinstance(ba, Modul), ob es überall dort eingesetzt werden kann, wo ein normales Modul erwartet wird – das funktioniert.

Design-Entscheidung: Die Werte ects=12 und name="Bachelorarbeit" sind im Konstruktor fest eingetragen – eine bewusste Vereinfachung für den Prototypen. Für spätere Erweiterungen (z. B. Masterstudiengang mit 30 ECTS) müssten diese Werte als Parameter übergeben werden.

Hinweis von Aiko Walte (Tutor): Die Abschlussarbeit darf laut Kardinalität aus Phase 1 (0..1) nur einmal pro Studiengang vorkommen. Python erzwingt das nicht von sich aus – die Prüfung gehört in die Klasse Studiengang, nicht in die Oberfläche. Es wäre unsauber, das erst in der Benutzeroberfläche zu prüfen, weil die Regel für das gesamte System gelten muss – egal ob die Daten über die App oder später z. B. per Datei-Import reinkommen. modul_hinzufuegen() prüft deshalb, ob bereits eine Abschlussarbeit vorhanden ist, und wirft bei Verstoß eine ValueError-Exception (phase2_test_komposition.py):

```
def modul_hinzufuegen(self, modul, semester):
    if isinstance(modul, Abschlussarbeit):
        alle = [m for s in self._semester for m in s.module]
        if any(isinstance(m, Abschlussarbeit) for m in alle):
            raise ValueError("Nur eine Abschlussarbeit pro Studiengang.")
    semester.modul_hinzufuegen(modul)
```

1.4 Komposition und Aggregation

In Python gibt es keinen speziellen Befehl für Komposition oder Aggregation – ich löse beides über Listen und Objektreferenzen (phase2_test_komposition.py). Im Code von 1.3 habe ich dafür auch eine List Comprehension eingesetzt, die alle Module über alle Semester abfragt – ein funktionales Konzept, das Python gut unterstützt. Der Unterschied zwischen Komposition und Aggregation liegt im Entwurf:

Komposition: Studiengang → Semester → Modul. Der Studiengang erzeugt das Semester selbst über erstelle_semester(), das Semester verwaltet seine Module intern. Wird der Studiengang gelöscht, verschwinden auch alle Semester und deren Module – der Lebenszyklus liegt vollständig beim jeweils übergeordneten Objekt. Das entspricht der Komposition, die im UML mit der ausgefüllten Raute (*--) dargestellt ist. Python hat dafür kein eigenes Sprachkonstrukt, der Effekt wird durch die interne Verwaltung der Objektlisten erreicht.

```
def erstelle_semester(self, nummer, startdatum, enddatum):
    sem = Semester(nummer, startdatum, enddatum)
    self._semester.append(sem)
    return sem  # nur zur Nutzung im selben Kontext
```

Aggregation: Modul → Prüfungsleistung. Eine Pruefungsleistung hängt nicht zwingend am Modul – das Modul kennt sie, ist aber nicht für sie zuständig. Anders als bei der Komposition verschwindet die Pruefungsleistung nicht automatisch, wenn das Modul wegfällt.

Fazit: Das Klassendiagramm aus Phase 1 konnte ich im Wesentlichen direkt übernehmen. Anpassungen gab es nur auf technischer Ebene: Getter und Setter habe ich durch @property ersetzt. Die Kardinalitätsprüfung für die Abschlussarbeit (0..1) habe ich explizit in modul_hinzufuegen() eingebaut, da Python das nicht automatisch erzwingt. An der fachlichen Struktur der Klassen selbst hat sich nichts geändert.

2. Gesamtarchitektur

Die fünf Entity-Klassen aus Phase 1 sind direkt als Python-Klassen umsetzbar. Für den Prototypen kommen drei weitere Bereiche hinzu: Persistenz, KPI-Berechnung und Oberfläche. Die Architektur gliedert sich in vier Pakete: Entity-Klassen, Domain-Services, Infrastruktur und UI.

2.1 Neue Klassen und ihre Aufgaben

JSONRepository: Lädt beim Programmstart die gespeicherten Daten aus einer JSON-Datei und schreibt Änderungen zurück. Diese Klasse ist bewusst von den Entity-Klassen getrennt: Studiengang muss nicht wissen, wie seine Daten gespeichert werden. Falls man die Speicherung später auf eine Datenbank umstellen will, muss nur JSONRepository angepasst werden – der Rest bleibt unberührt.

KPICalculator: Berechnet die vier Dashboard-Kennzahlen auf Basis des Studiengang-Objekts. Ich habe die Berechnungen ausgelagert, damit die Studiengang-Klasse nicht zu groß wird – Studiengang speichert die Daten, KPICalculator rechnet daraus, was das Dashboard anzeigen soll.

DashboardApp: Zentrale tkinter-Klasse und Einstiegspunkt der Anwendung. Sie hält Referenzen auf Studiengang, JSONRepository und KPICalculator und verbindet damit alle Teile der Anwendung. Sie baut KPI-Leiste, Modultabelle und Navigation auf und gibt Benutzeraktionen an ModulDialog oder EingabeDialog weiter. Die Dialoge sind bewusst ausgelagert, damit DashboardApp nicht zu viele Aufgaben übernimmt.

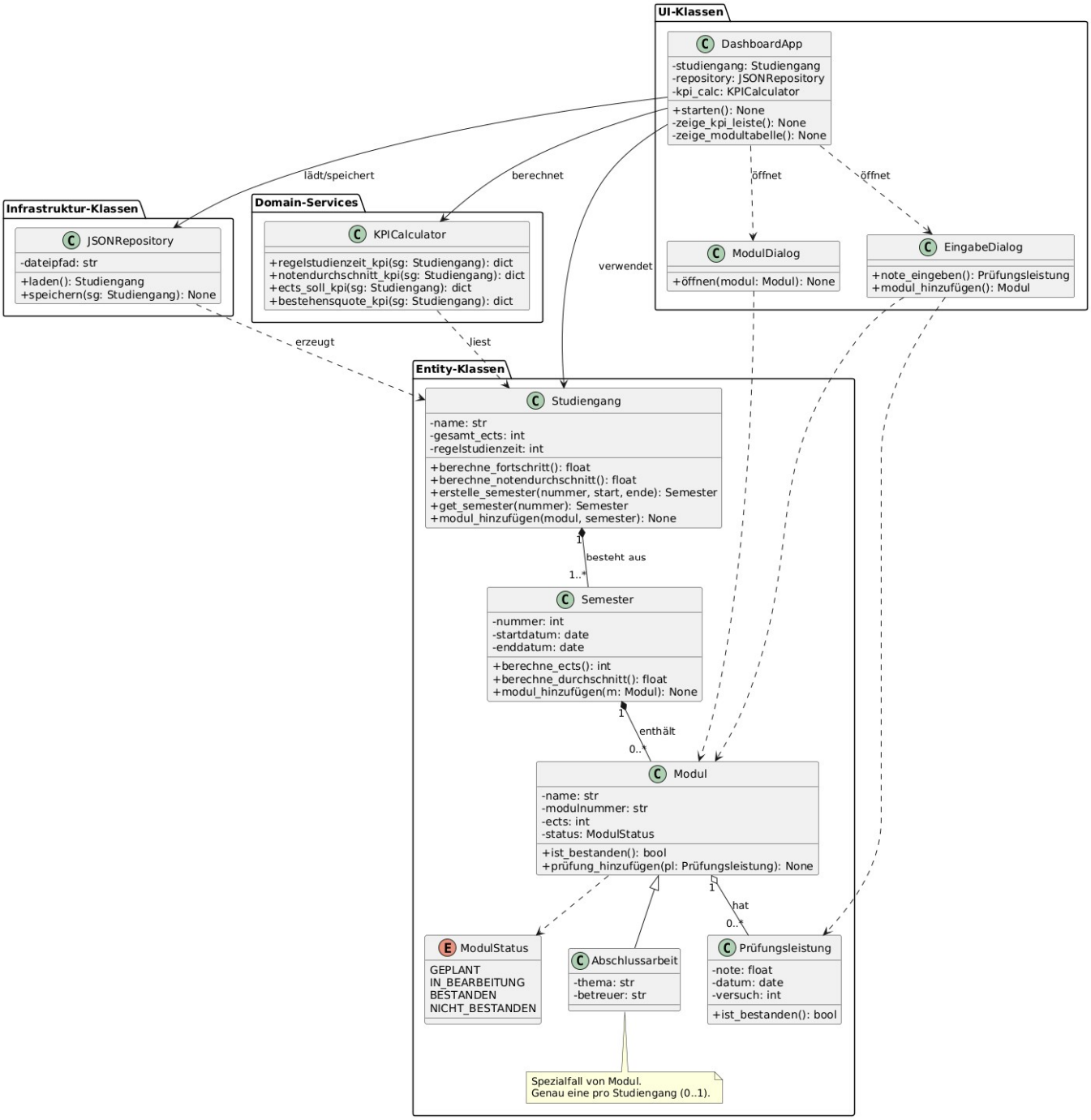
ModulDialog: Öffnet ein Detailfenster für ein einzelnes Modul und zeigt alle zugehörigen Prüfungsversuche an. Die Klasse ist bewusst von DashboardApp getrennt, damit die Hauptansicht übersichtlich bleibt.

EingabeDialog: Übernimmt die Eingabe neuer Module und Noten über ein eigenes Fenster. Die erzeugten Objekte werden an DashboardApp zurückgegeben, die sie in den Studiengang einfügt und über JSONRepository speichert. Auch hier gilt: Die Eingabelogik gehört nicht in die Hauptklasse.

2.2 UML-Klassendiagramm Gesamtarchitektur

Das Diagramm wurde mit PlantUML erstellt und zeigt alle Klassen in vier Paketen. Die wichtigste Entwurfsentscheidung dabei: Die Abhängigkeiten verlaufen immer von außen nach innen. Entity-Klassen kennen weder Repository noch Oberfläche – sie wissen nur, was fachlich zu ihnen gehört. DashboardApp hält dauerhafte Verbindungen zu Studiengang, JSONRepository und KPICalculator (durchgezogene Pfeile); die Dialoge werden nur bei Bedarf geöffnet und danach wieder geschlossen (gestrichelte Pfeile).

Die verwendeten Beziehungsnotationen im Überblick: Ausgefüllte Rauten kennzeichnen Kompositionen (Studiengang → Semester, Semester → Modul), leere Rauten Aggregationen (Modul → Prüfungsleistung), und der Pfeil mit leerem Dreieck zeigt die Vererbungsbeziehung (Abschlussarbeit → Modul). Durchgezogene Pfeile stellen dauerhafte Assoziationen dar;

gestrichelte Pfeile (Use-Dependency) kennzeichnen temporäre Nutzungsbeziehungen, etwa die Dialoge, die von DashboardApp nur bei Bedarf geöffnet werden.

Hinweis zur Benennung: Das UML-Diagramm verwendet deutsche Bezeichner mit Umlauten (z. B. Prüfungsleistung), weil das fachlich passt. Im Python-Code sind Umlaute in Bezeichnern zwar technisch möglich, aber unüblich – deshalb steht dort z. B. Pruefungsleistung. Die Klassen und Methoden sind trotzdem dieselben.